

Data Structures

Lecture 11: BST & AVL Tree

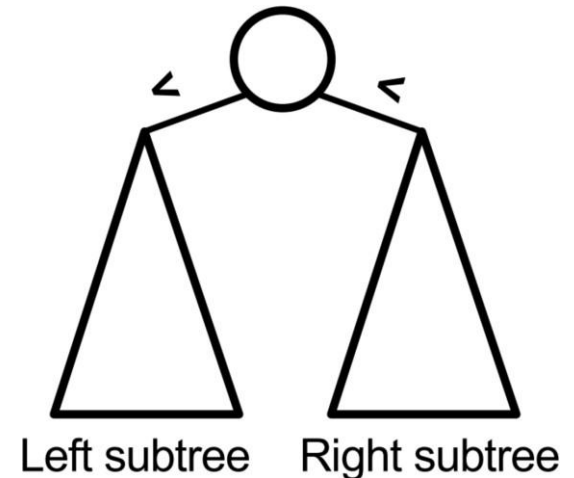
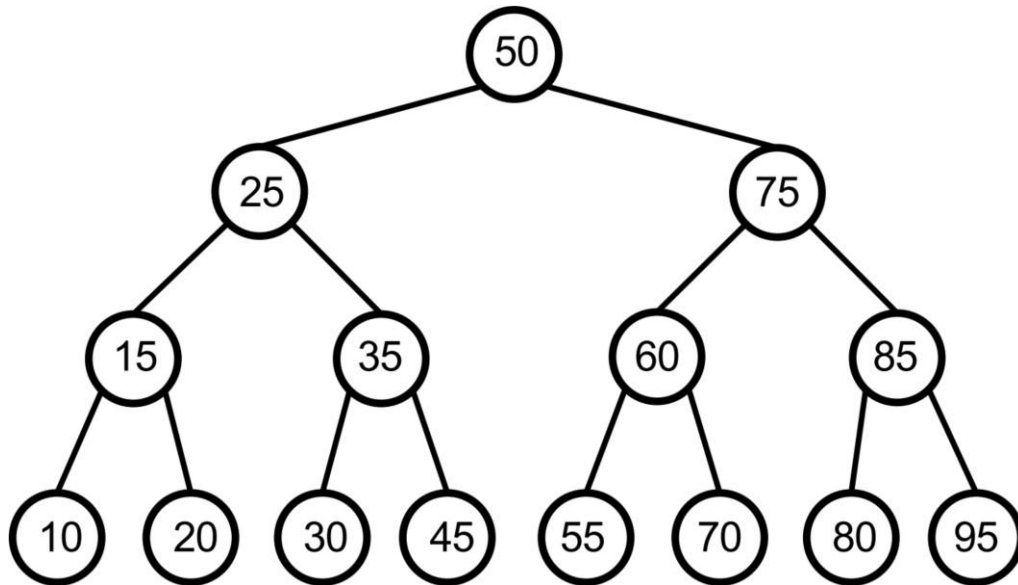
Soobin Um

Apr. 8, 2026

복습

Definition: Binary Search Trees

- 이진 탐색 트리(BST): 다음 조건을 따르는 이진 트리
 - 모든 노드는 **유일한 키**를 가짐
 - K라는 키를 갖는 노드의 **왼쪽 subtree**에 있는 모든 노드는 **K보다 작은 키**를 가짐
 - K라는 키를 갖는 노드의 **오른쪽 subtree**에 있는 모든 노드는 **K보다 큰 키**를 가짐
 - 모든 subtree는 BST임



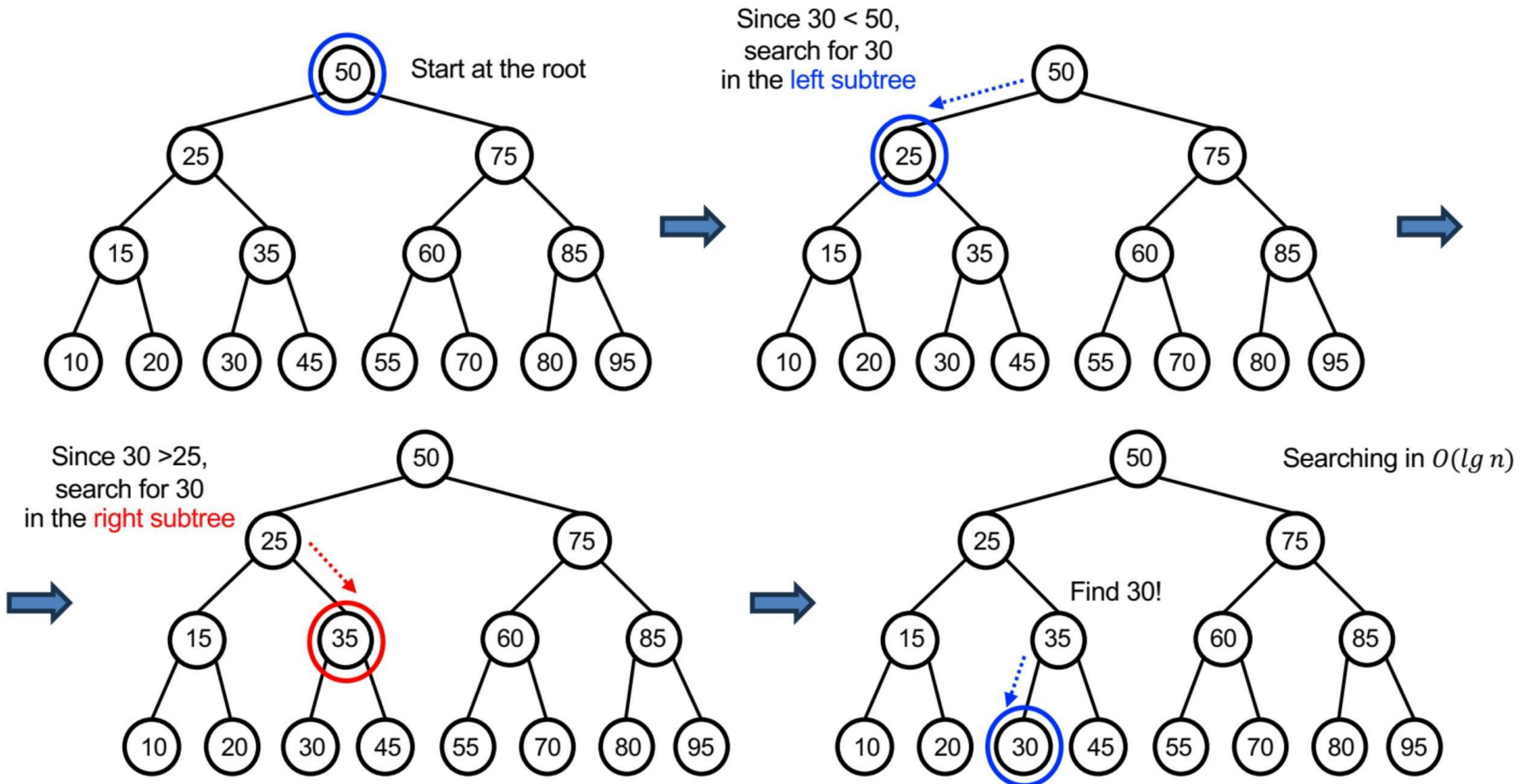
ADT: Binary Search Tree

값에 대한 비교가 가능한 타입만이 key로써 사용되도록 제약

```
public interface BST<Key extends Comparable<Key>, E> {  
    /** Reinitialize BST: remove all records */  
    public void clear();  
  
    /** Insert a record with key k and element e */  
    public void insert(Key k, E e);  
  
    /** Remove and return the record matching key k (null if not found) */  
    public E remove(Key k);  
  
    /** Remove and return the record with the smallest key (null if tree is empty) */  
    public E removeMin();  
  
    /** Remove and return the record with the largest key (null if tree is empty) */  
    public E removeMax();  
  
    /** Return a record matching key k (null if not found) */  
    public E find(Key k);  
  
    /** @return The number of records currently stored in the BST */  
    public int size();  
};
```

Operations: Search

- Ex) Key가 30인 노드 찾기



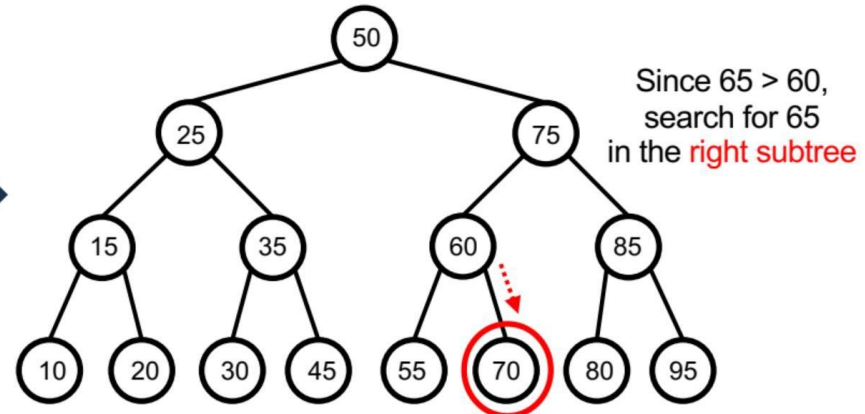
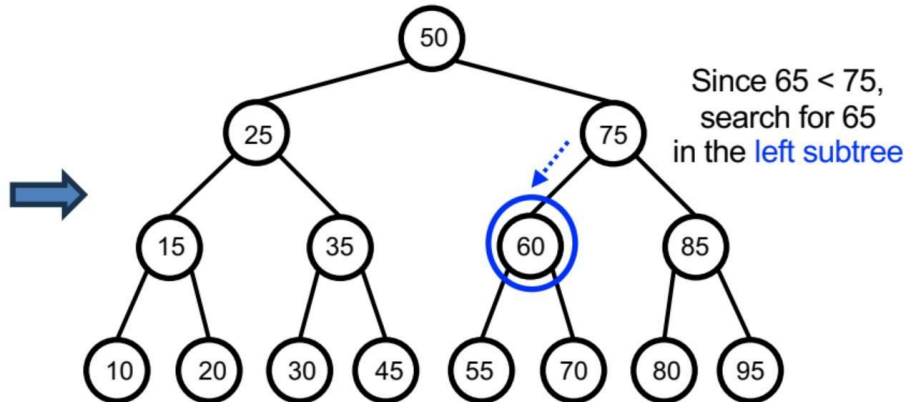
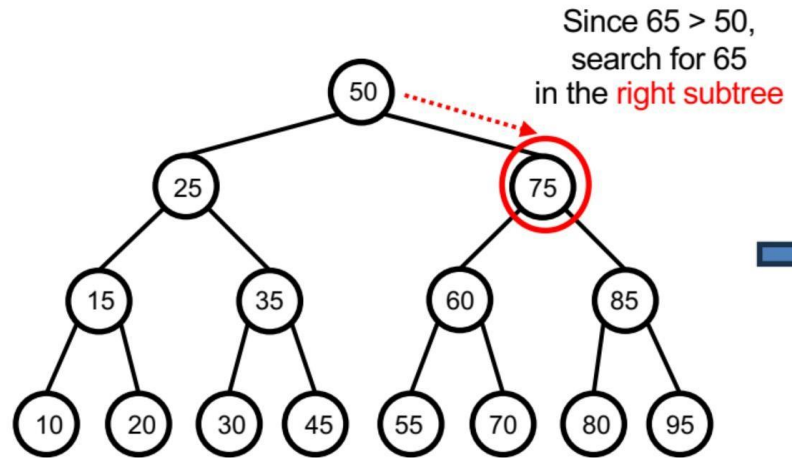
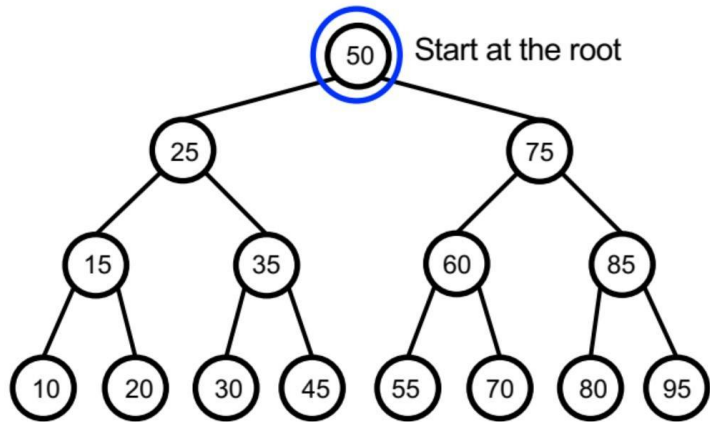
Operations: Search (코드)

```
/** Return a record matching key k (null if not found) */
public E find(Key k) {      // 외부에서 호출하는 메서드
    return findHelper(root, k);    // 트리 최상단 노드(root)부터 탐색 시작
}

// 실제 탐색을 수행하는 재귀 메서드
private E findHelper(Node rt, Key k) {
    if (rt == null) return null;
    if (rt.key.compareTo(k) == 0)    // rt.key == k
        return rt.element;
    else if (rt.key.compareTo(k) > 0)    // rt.key > k
        return findHelper(rt.left, k);
    else    // rt.key < k
        return findHelper(rt.right, k);
}
```

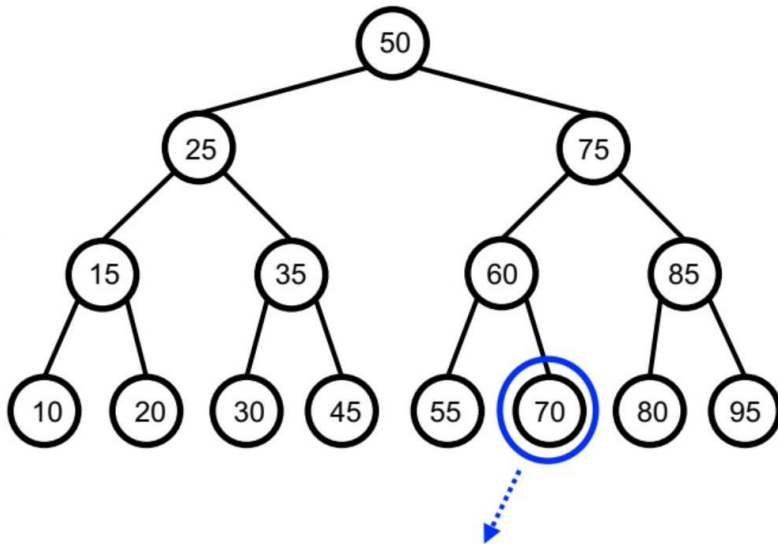
Operations: Insert

- Ex) Key가 65인 노드 넣기

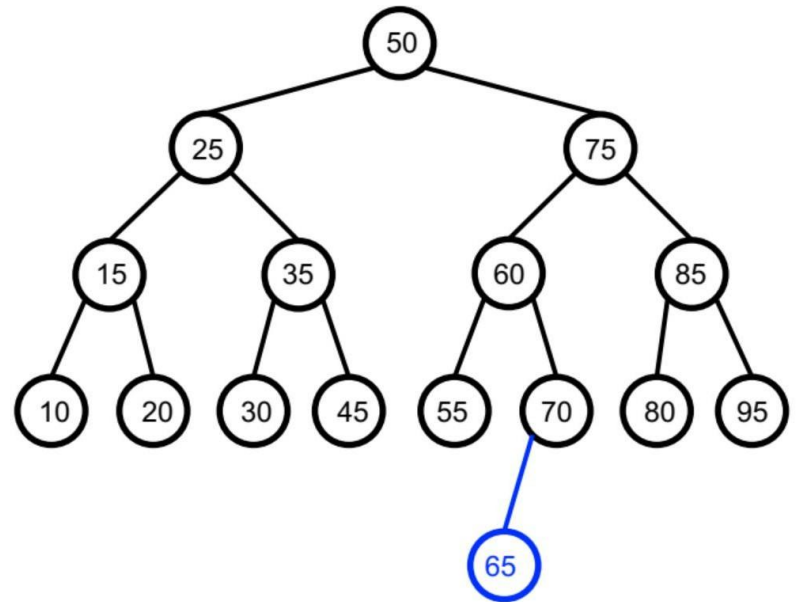


Operations: Insert

- Ex) Key가 65인 노드 넣기



Since $65 < 70$,
search for 65
in the **left subtree**



Since '70' has no child,
Insert '65' as the **left child** of the node 70

Operations: Insert (코드)

```
/** Insert a record with key k and element e */
public void insert(Key k, E e) {
    root = insertHelper(root, k, e);
    nodeCount++;
}

private Node insertHelper(Node rt, Key k, E e) {
    if (rt == null) return new Node(k, e); // 빈 자리 찾음; 새 노드 삽입

    int cmp = rt.key.compareTo(k);
    if (cmp > 0) // rt.key > k
        rt.left = insertHelper(rt.left, k, e);
    else if (cmp < 0) // rt.key <= k
        rt.right = insertHelper(rt.right, k, e);
    else // rt.key == k
        rt.element = e; // 기존 값 덮어쓰임
    return rt;
}
```

최종적으로 항상 전체 트리의 루트 노드가 반환됨

(새로운 노드 삽입 후, 재귀호출이 끝나면서 각 단계에서의 서브트리의 루트가 반환.
이 과정이 재귀적으로 반복됨으로써, 최종적으로 전체 트리의 루트가 반환됨)

Binary Search Trees

Operations: Delete

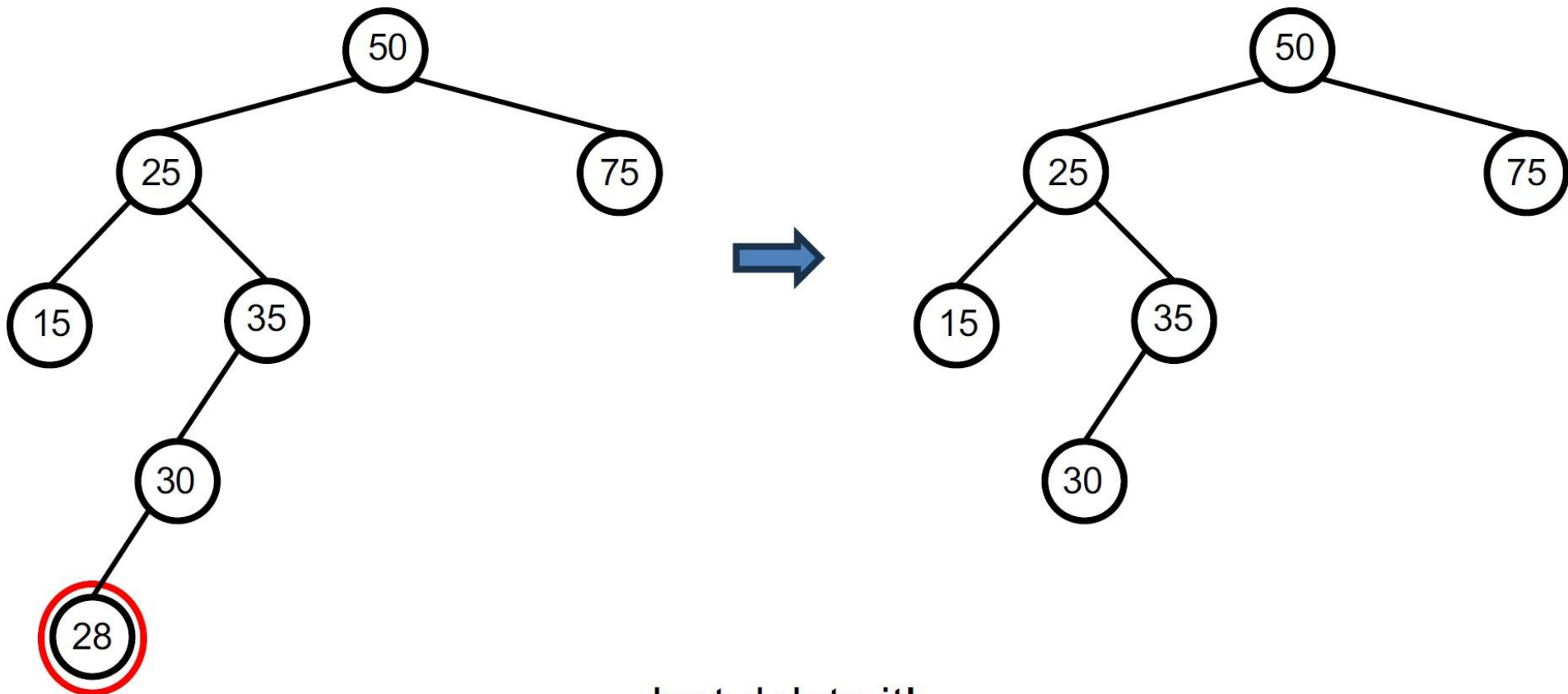
- Key가 x 인 노드 삭제
 - (1) 삭제 대상이 되는 (즉, key가 x 인) 노드를 탐색 (search)

Operations: Delete

- Key가 x 인 노드 삭제
 - (1) 삭제 대상이 되는 (즉, key가 x 인) 노드를 탐색 (search)
 - (2) 다음의 세 가지 케이스를 구분
 - (a) 삭제 대상 노드가 자식 노드를 갖지 않는 경우
 - (b) 삭제 대상 노드가 하나의 자식 노드를 갖는 경우
 - (c) 삭제 대상 노드가 두 개의 자식 노드들을 갖는 경우

Operations: Delete

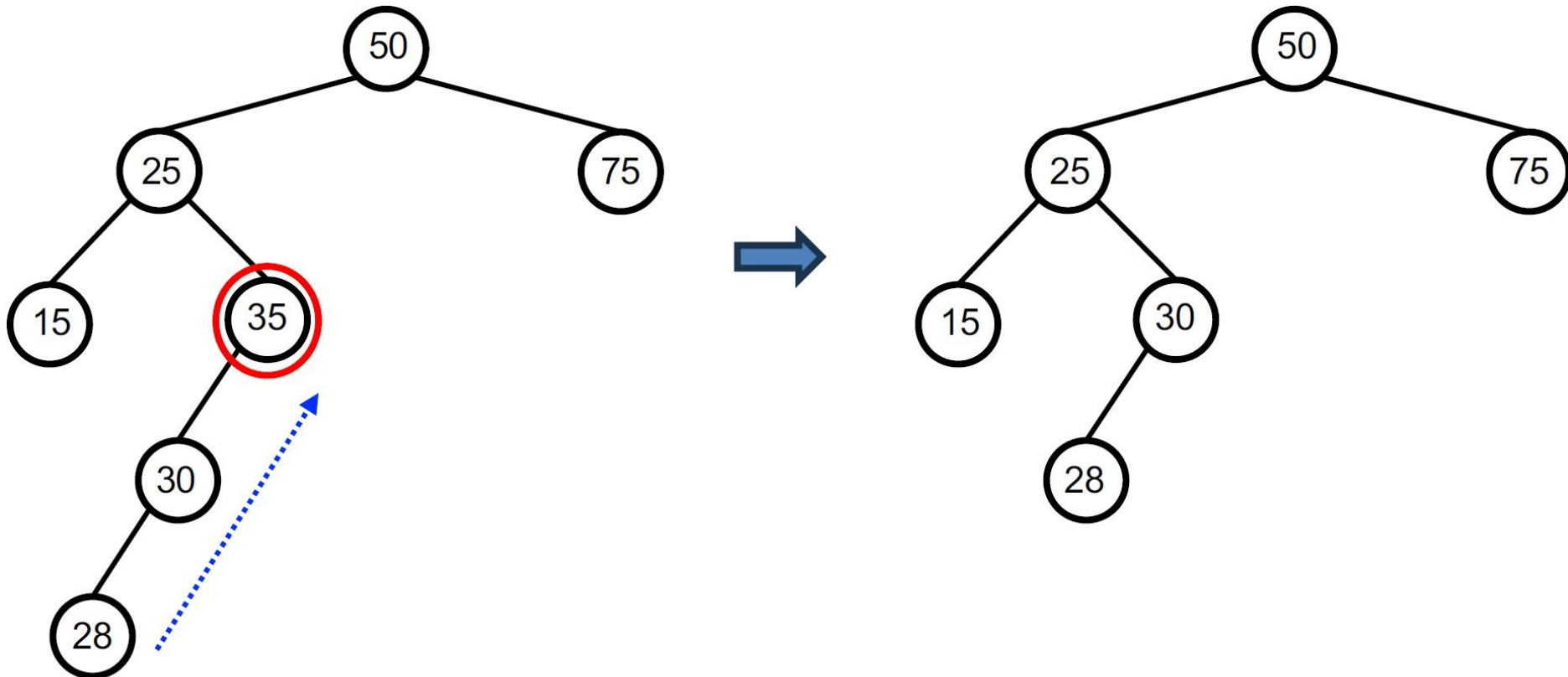
- (a) 삭제 대상 노드가 자식 노드를 갖지 않는 경우
 - 단순 삭제 수행
 - 해당 노드의 부모 노드가 해당 노드를 가리키는 포인터 값을 null 값으로 갖게 됨



Just delete it!

Operations: Delete

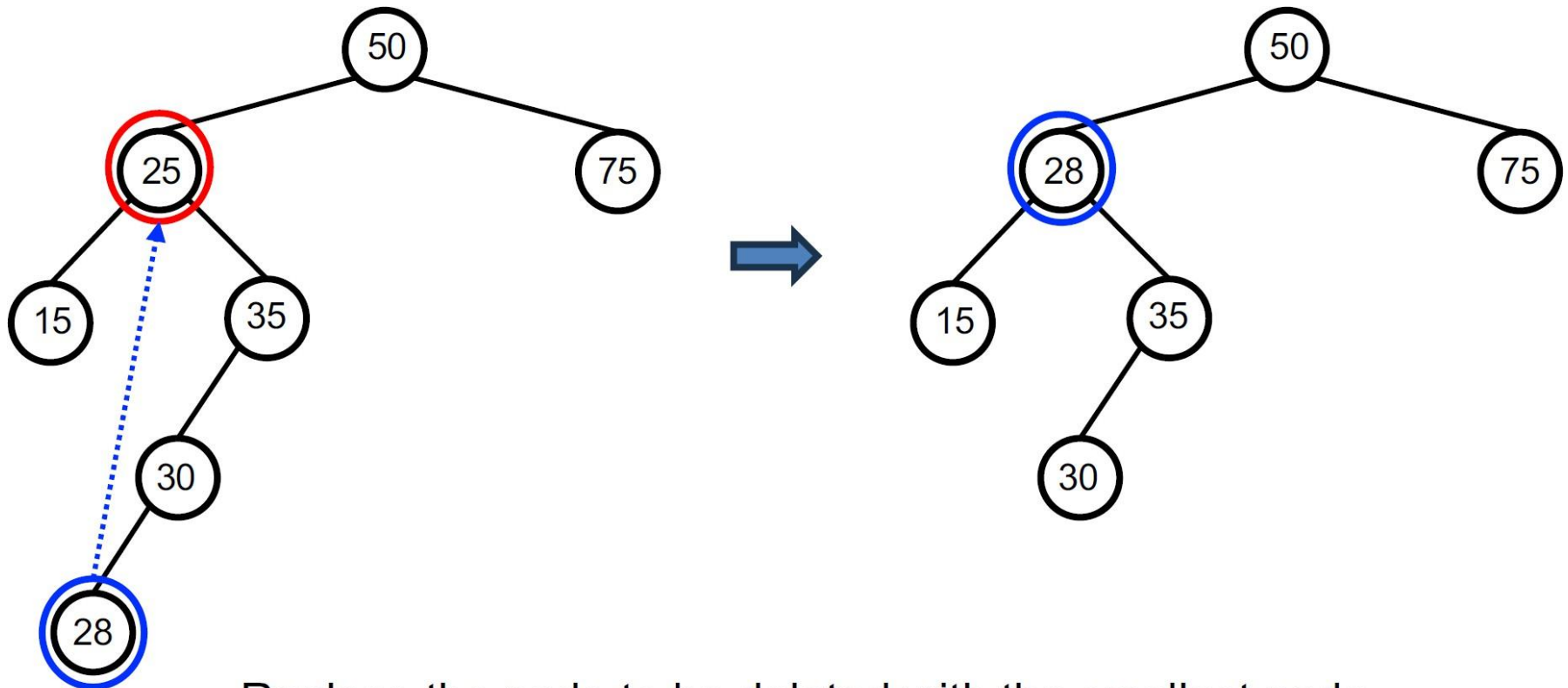
- (b) 삭제 대상 노드가 하나의 자식 노드를 갖는 경우
 - 삭제 대상 노드를 해당 노드의 **자식 노드로 대체**
 - 해당 노드의 자식 노드를 (해당 노드의) 부모 노드에 연결



Delete the node and plug the child into the spot
where the deleted node was

Operations: Delete

- (c) 삭제 대상 노드가 두 개의 자식 노드들을 갖는 경우
 - 삭제 대상 노드를 해당 노드의 **오른쪽 서브트리의 최소 노드로 대체**
 - 즉, **해당 노드보다 큰 key 값을 갖는 노드들 중 가장 작은 값의 key를 갖는 노드**



Replace the node to be deleted with the smallest node in the right subarray (i.e., successor)

Operations: Delete (코드)

```
/** Remove and return the record matching key k (null if not found) */
public E remove(Key k) {
    E[] temp = (E[]) new Object[1]; // 삭제된 노드의 element 값을 담을 길이 1짜리 배열
    root = removeHelper(root, k, temp);

    if (temp[0] != null)
        nodeCount--;

    return temp[0]; // 삭제된 노드의 element 값을 반환
}
```


Operations: Delete (코드)

```
private Node removeHelper(Node rt, Key k, E[] elem) {
    if (rt == null) return null;

    if (rt.key.compareTo(k) > 0)    // rt.key > k
        rt.left = removeHelper(rt.left, k, elem);

    else if (rt.key.compareTo(k) < 0)    // rt.key < k
        rt.right = removeHelper(rt.right, k, elem);

    else {    // rt.key == k (즉, key k에 해당되는 노드를 찾은 경우)
        elem[0] = rt.element;
        /* (a),(b) rt의 오른쪽 자식 노드를 rt의 부모 노드와 연결 */
        if (rt.left == null) return rt.right;
        /* (b) rt의 왼쪽 자식 노드를 rt의 부모 노드와 연결 */
        else if (rt.right == null) return rt.left;
        /* (c) rt가 두 개의 자식 노드들을 갖는 경우 */
        else {
            Node minNode = getMin(rt.right);    // 오른쪽 서브트리의 최소 노드 찾기
            /* 최소 노드의 key와 element를 삭제 대상 노드의 key와 element에 각각 복사 */
            rt.key = minNode.key;
            rt.element = minNode.element;
            rt.right = deleteMin(rt.right);    // 최소 노드 삭제
        }
    }
    return rt;
}
```

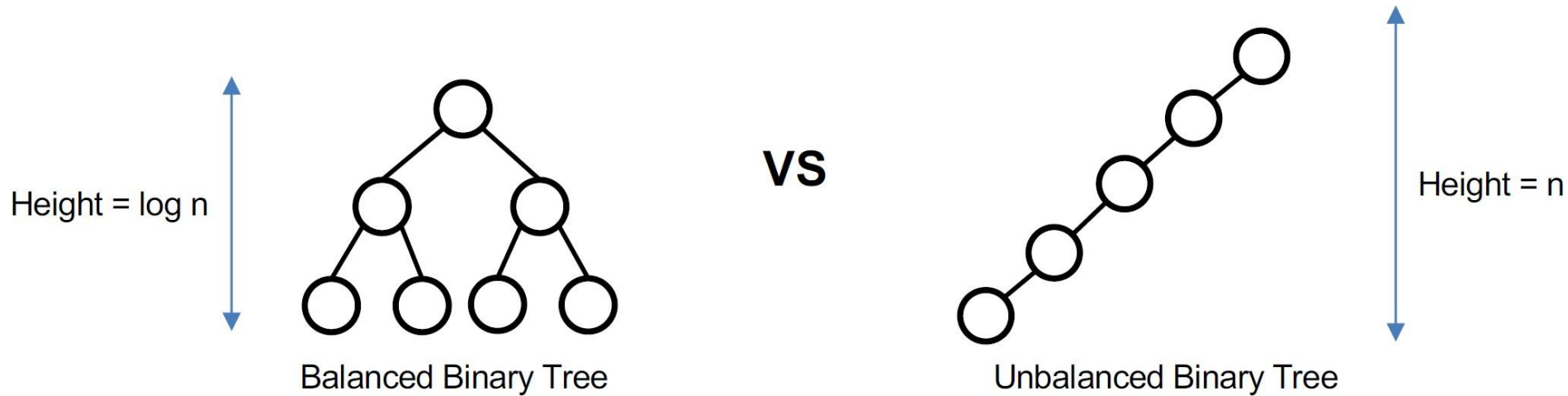
Operations: Delete (코드)

```
private Node getMin(Node rt) {  
    if (rt.left == null) return rt;  
  
    return getMin(rt.left);  
}
```

```
private Node deleteMin(Node rt) {  
    if (rt.left == null) return rt.right;    // 최소값 노드 삭제(오른쪽 자식 반환)  
  
    rt.left = deleteMin(rt.left);  
    return rt;  
}
```

Cost Analysis

- BST 연산의 시간 복잡도는 트리의 높이(height)에 따라 달라짐
 - Best case: $O(\log n)$ – 완전이진트리 또는 균형트리인 경우
 - Worst case: $O(n)$ – 편향(skewed)트리인 경우

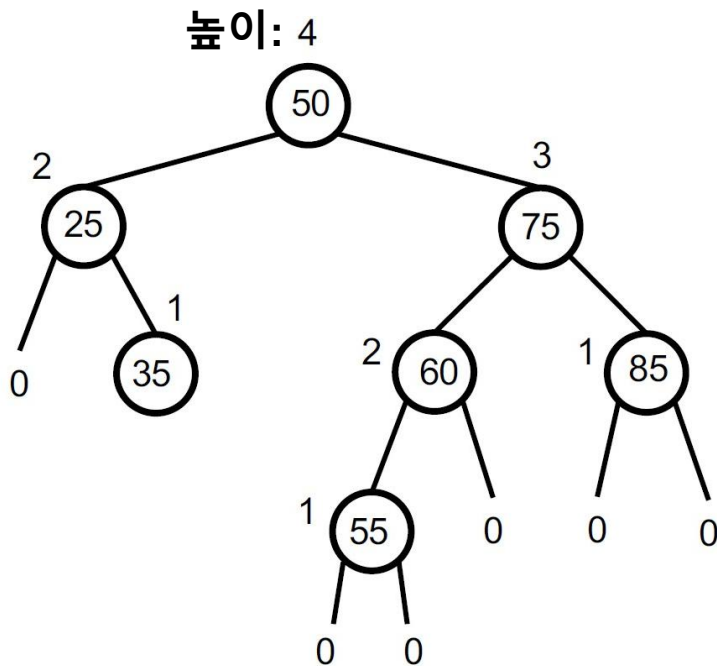


BST가 높은 효율성을 갖기 위해선, 균형 잡힌 트리 구조가 필수적

AVL Trees

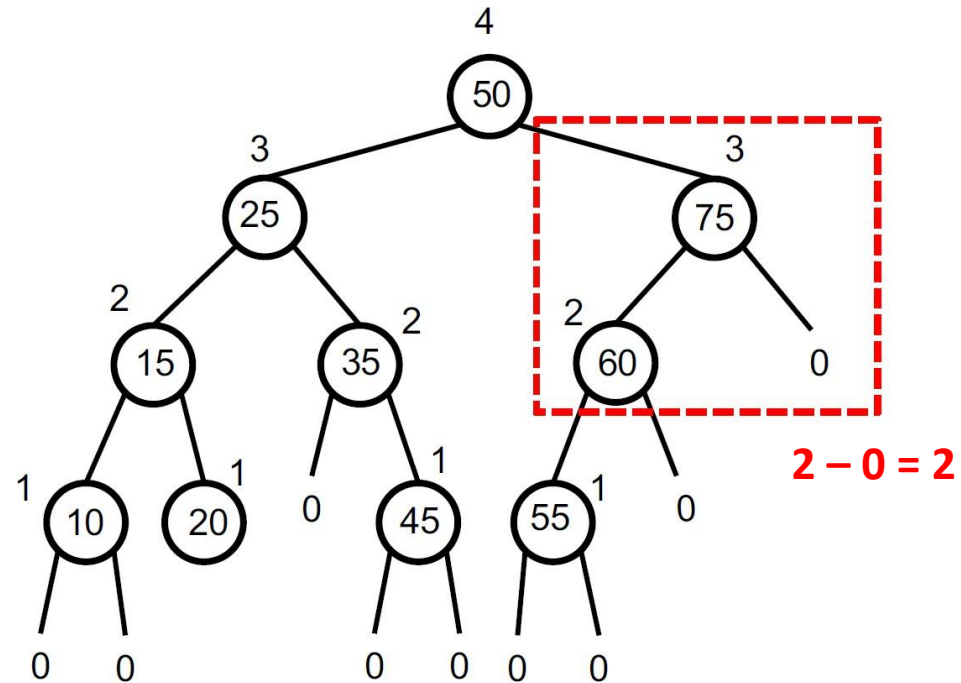
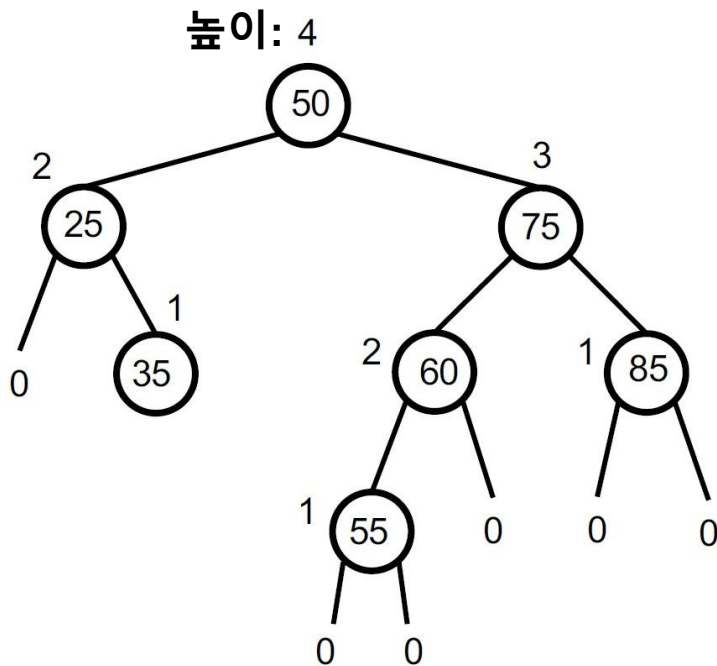
AVL (Adelson-Velsky and Landis) Tree

- 모든 노드에서 왼쪽과 오른쪽 서브트리 간의 **높이 차이가 최대 1을 넘지 않도록 보장**하는 균형 이진 탐색 트리 (**self-balancing BST**)
 - Balance Factor: 왼쪽 서브트리와 오른쪽 서브트리의 높이 차이 (즉, 균형 상태를 수치화한 값)



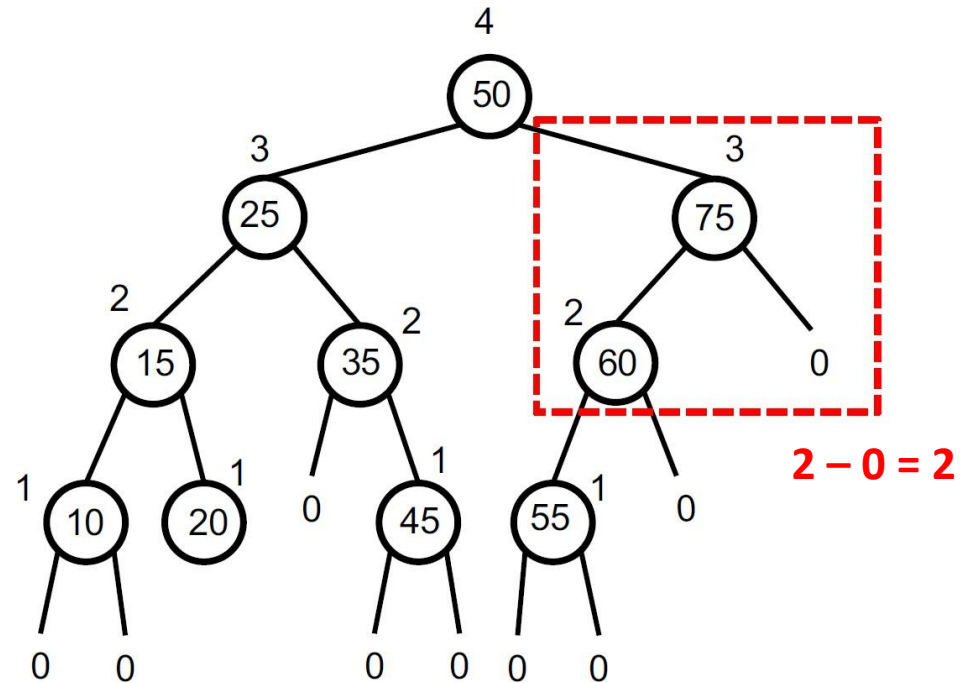
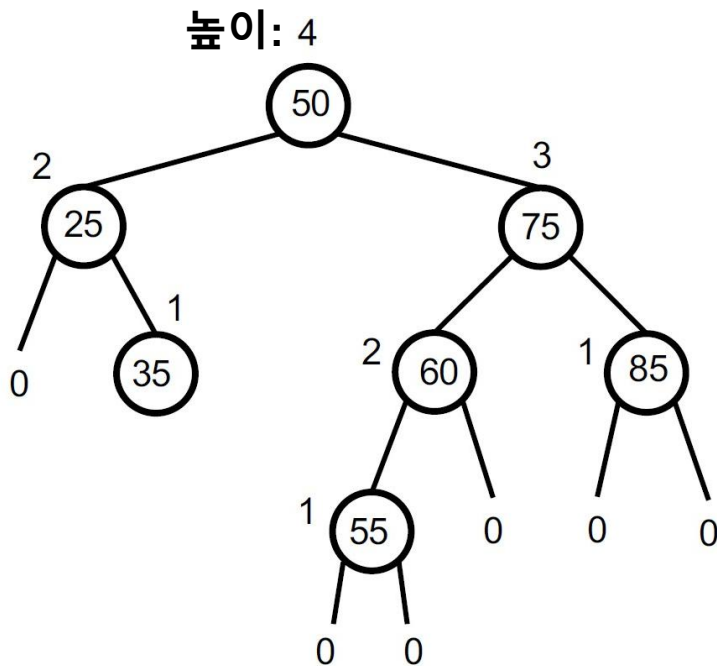
AVL (Adelson-Velsky and Landis) Tree

- 모든 노드에서 왼쪽과 오른쪽 서브트리 간의 **높이 차이가 최대 1을 넘지 않도록 보장**하는 균형 이진 탐색 트리 (**self-balancing BST**)
 - Balance Factor: 왼쪽 서브트리와 오른쪽 서브트리의 높이 차이 (즉, 균형 상태를 수치화한 값)



AVL (Adelson-Velsky and Landis) Tree

- 모든 노드에서 왼쪽과 오른쪽 서브트리 간의 **높이 차이가 최대 1을 넘지 않도록 보장**하는 균형 이진 탐색 트리 (**self-balancing BST**)
 - Balance Factor: 왼쪽 서브트리와 오른쪽 서브트리의 높이 차이 (즉, 균형 상태를 수치화한 값)



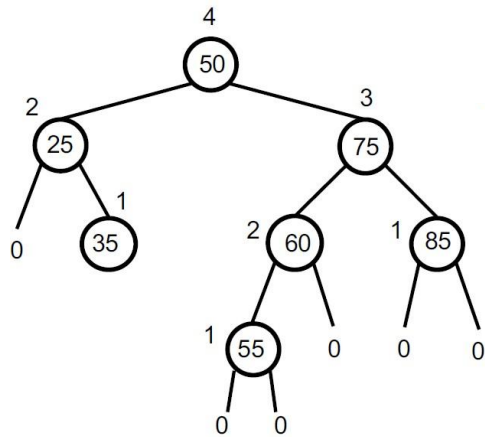
AVL Tree 조건 위반

Insertion on AVL Tree

- Key가 w 인 노드 삽입
 - (1) 일반적인 BST에서의 삽입 연산을 통해 key가 w 인 노드를 삽입
 - (2) 삽입된 w 노드에서부터 위로 올라가며 불균형이 발생하는 첫 번째 노드 z 를 찾기
 - (3) 노드 z 를 루트로 갖는 서브트리 상에서의 “적절한 회전”을 통해, 균형 맞춤

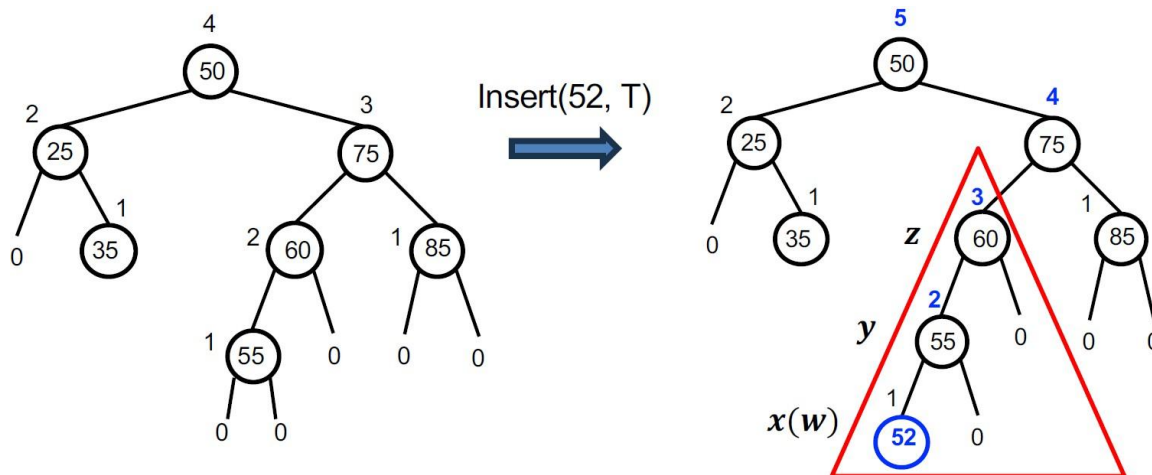
Insertion on AVL Tree

- Key가 w 인 노드 삽입
 - (1) 일반적인 BST에서의 삽입 연산을 통해 key가 w 인 노드를 삽입
 - (2) 삽입된 w 노드에서부터 위로 올라가며 불균형이 발생하는 첫 번째 노드 z 를 찾기
 - (3) 노드 z 를 루트로 갖는 서브트리 상에서의 “적절한 회전”을 통해, 균형 맞춤



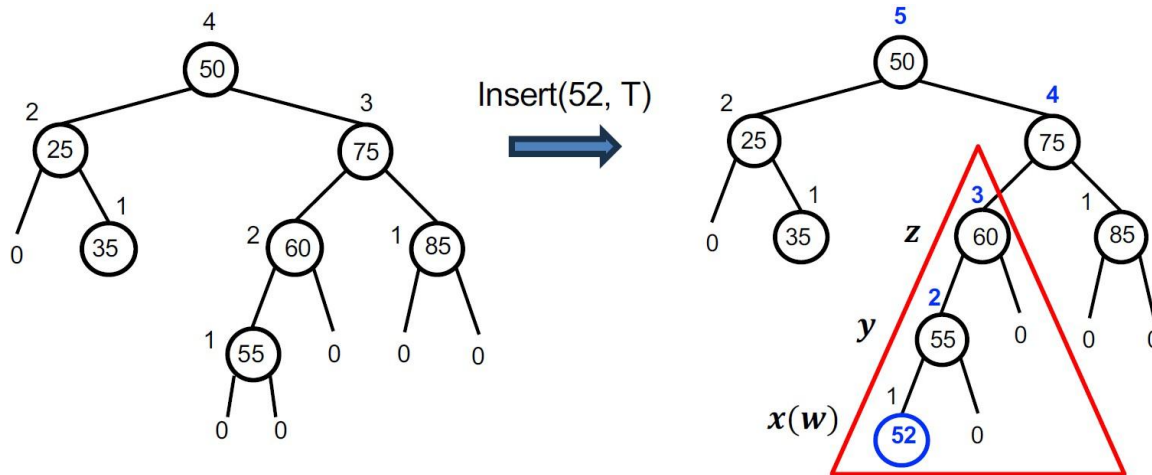
Insertion on AVL Tree

- Key가 w 인 노드 삽입
 - (1) 일반적인 BST에서의 삽입 연산을 통해 key가 w 인 노드를 삽입
 - (2) 삽입된 w 노드에서부터 위로 올라가며 불균형이 발생하는 첫 번째 노드 z 를 찾기
 - (3) 노드 z 를 루트로 갖는 서브트리 상에서의 “적절한 회전”을 통해, 균형 맞춤



Insertion on AVL Tree

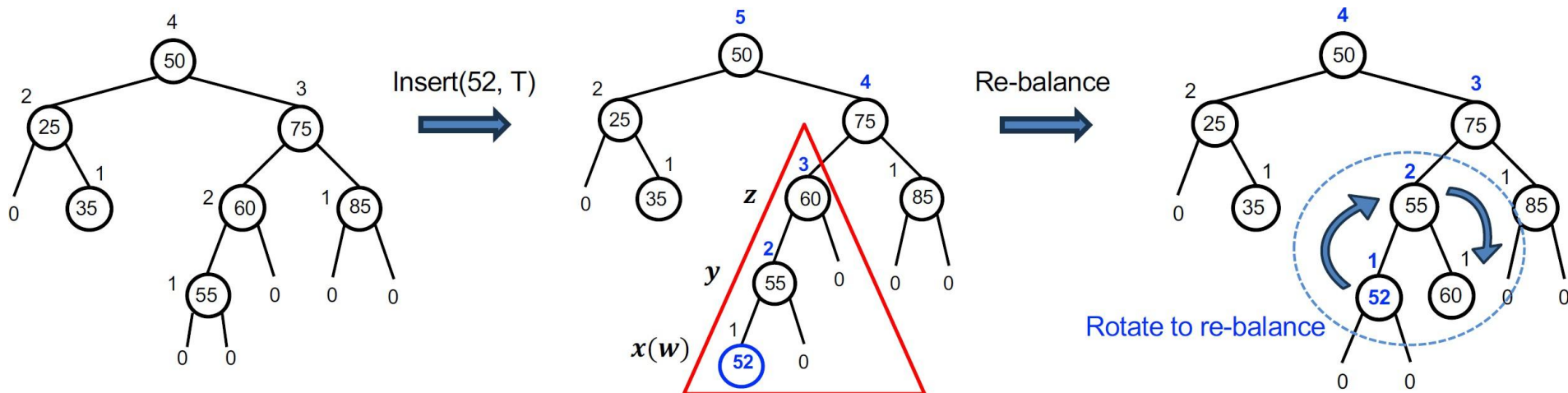
- Key가 w 인 노드 삽입
 - (1) 일반적인 BST에서의 삽입 연산을 통해 key가 w 인 노드를 삽입
 - (2) 삽입된 w 노드에서부터 위로 올라가며 불균형이 발생하는 첫 번째 노드 z 를 찾기
 - (3) 노드 z 를 루트로 갖는 서브트리 상에서의 “적절한 회전”을 통해, 균형 맞춤



새로 삽입된 노드의 조상 중 하나인 노드 z 에서 불균형 발생
($2 - 0 = 2$)

Insertion on AVL Tree

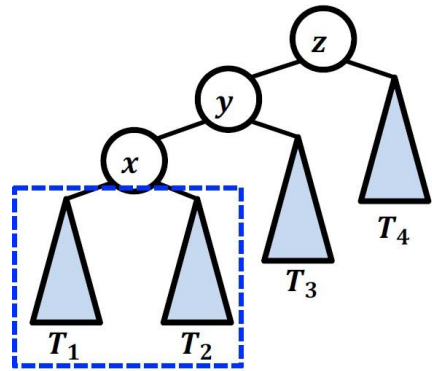
- Key가 w 인 노드 삽입
 - (1) 일반적인 BST에서의 삽입 연산을 통해 key가 w 인 노드를 삽입
 - (2) 삽입된 w 노드에서부터 위로 올라가며 불균형이 발생하는 첫 번째 노드 z 를 찾기
 - (3) 노드 z 를 루트로 갖는 서브트리 상에서의 “적절한 회전”을 통해, 균형 맞춤



새로 삽입된 노드의 조상 중 하나인 노드 z 에서 불균형 발생
($2 - 0 = 2$)

Insertion on AVL Tree

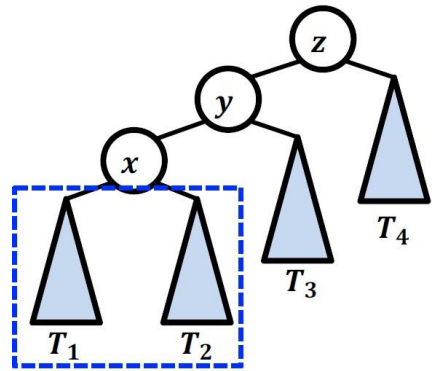
- (노드 z 에서 발생하는) 4가지 불균형 케이스
 - (Case 1) LL: 노드 z 의 왼쪽 자식의 왼쪽 서브트리 내 삽입



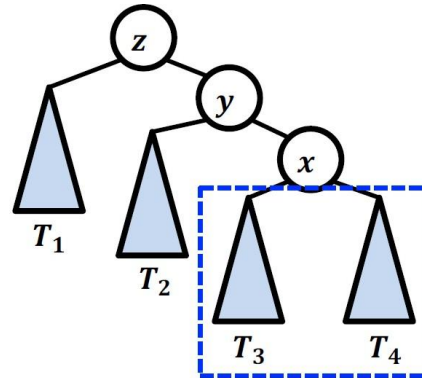
Case 1: LL

Insertion on AVL Tree

- (노드 z 에서 발생하는) 4가지 불균형 케이스
 - (Case 1) LL: 노드 z 의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - (Case 2) RR: 노드 z 의 오른쪽 자식의 오른쪽 서브트리 내 삽입



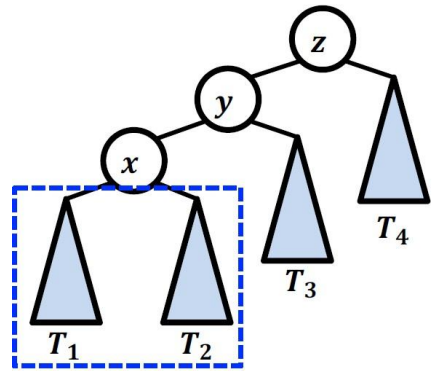
Case 1: LL



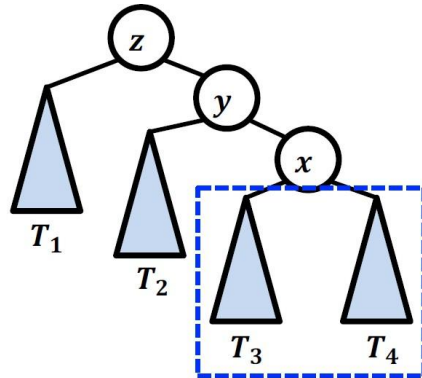
Case 2: RR

Insertion on AVL Tree

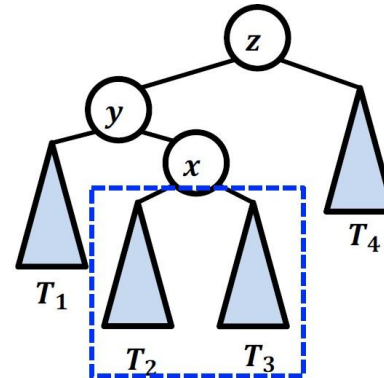
- (노드 z에서 발생하는) 4가지 불균형 케이스
 - (Case 1) LL: 노드 z의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - (Case 2) RR: 노드 z의 오른쪽 자식의 오른쪽 서브트리 내 삽입
 - (Case 3) LR: 노드 z의 왼쪽 자식의 오른쪽 서브트리 내 삽입



Case 1: LL



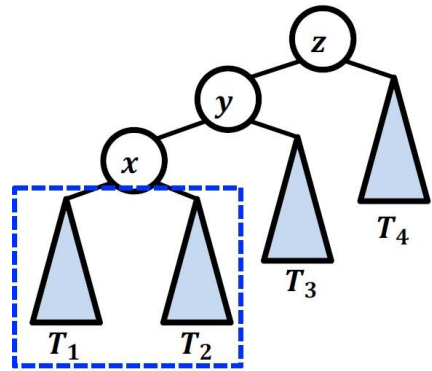
Case 2: RR



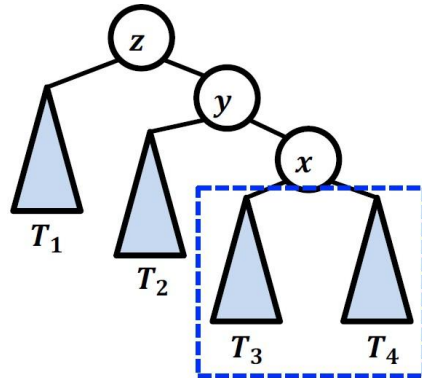
Case 3: LR

Insertion on AVL Tree

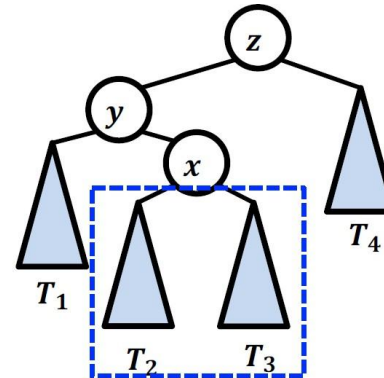
- (노드 z에서 발생하는) 4가지 불균형 케이스
 - (Case 1) LL: 노드 z의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - (Case 2) RR: 노드 z의 오른쪽 자식의 오른쪽 서브트리 내 삽입
 - (Case 3) LR: 노드 z의 왼쪽 자식의 오른쪽 서브트리 내 삽입
 - (Case 4) RL: 노드 z의 오른쪽 자식의 왼쪽 서브트리 내 삽입



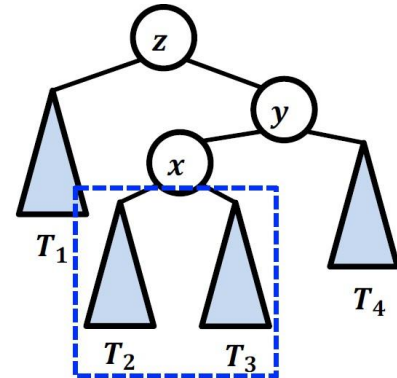
Case 1: LL



Case 2: RR



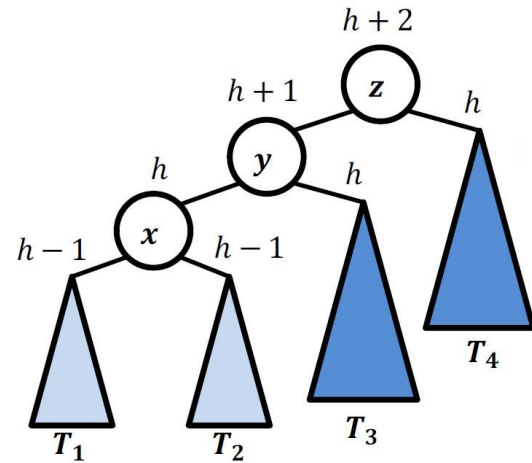
Case 3: LR



Case 4: RL

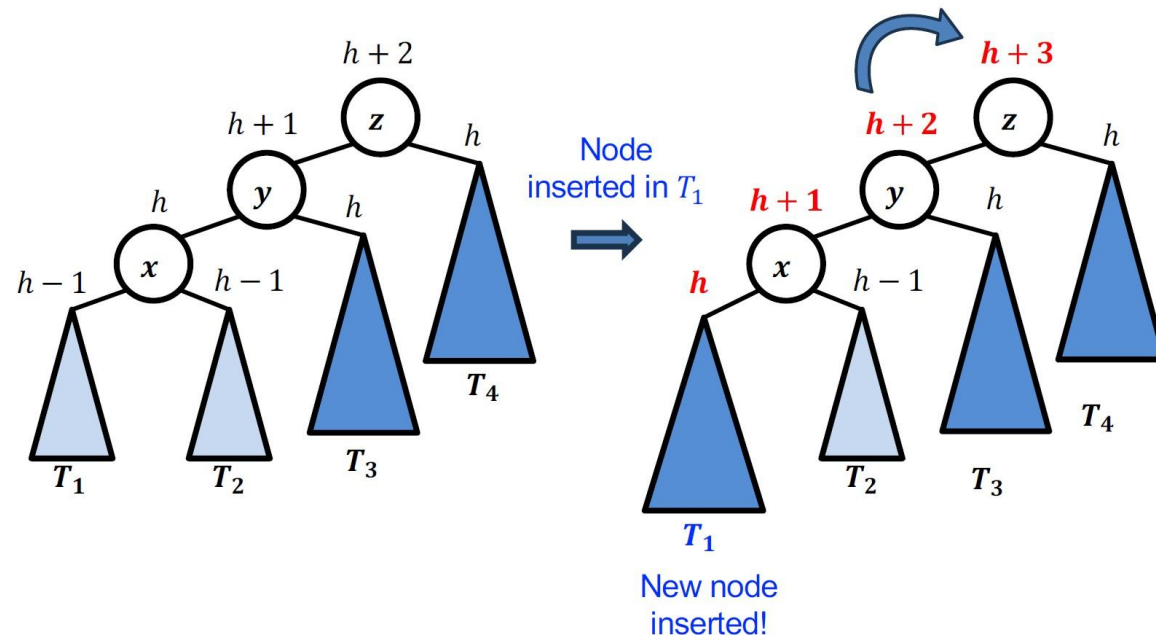
Insertion on AVL Tree (Case 1)

- **(Case 1) LL:** 노드 z 의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 z 에서의 “Right Rotation”을 통해 균형 맞춤



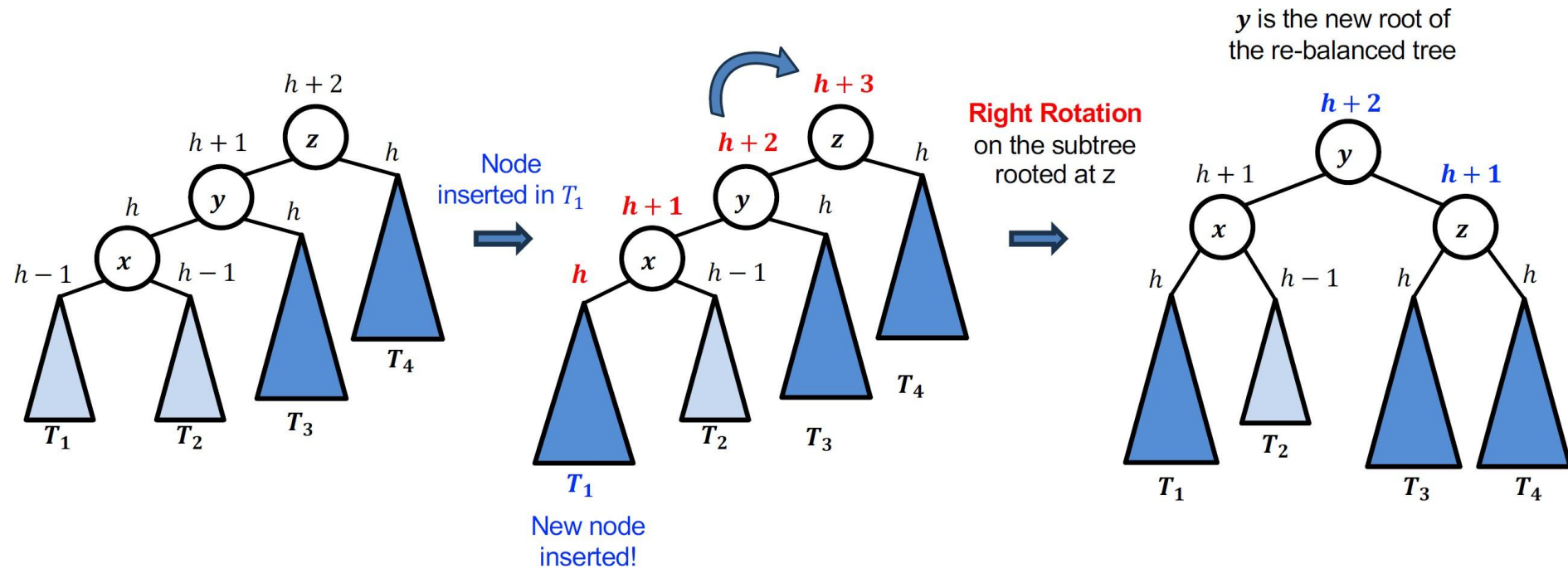
Insertion on AVL Tree (Case 1)

- **(Case 1) LL:** 노드 z 의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 z 에서의 “Right Rotation”을 통해 균형 맞춤



Insertion on AVL Tree (Case 1)

- **(Case 1) LL:** 노드 z 의 왼쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 z 에서의 “Right Rotation”을 통해 균형 맞춤

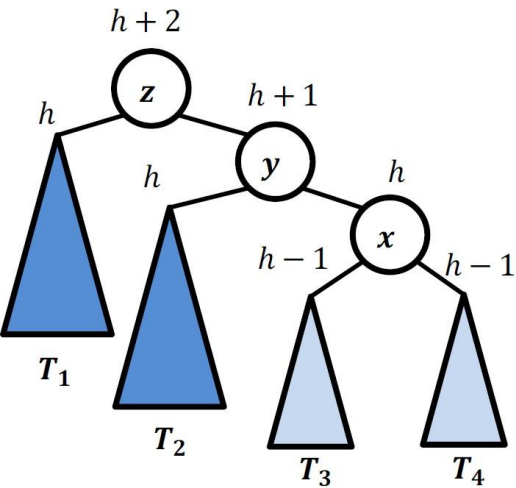


Right Rotation:

노드 z 가 y 의 오른쪽 자식 노드가 되며,
 y 의 기존 오른쪽 자식 노드는 z 의 왼쪽 자식 노드가 됨

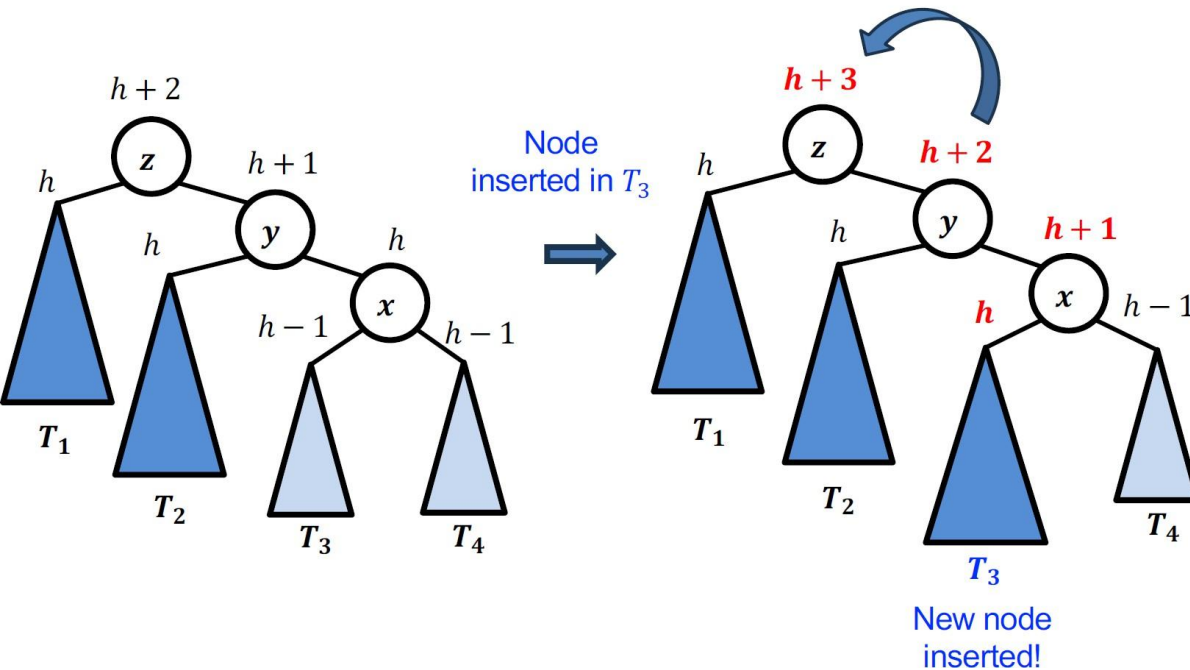
Insertion on AVL Tree (Case 2)

- **(Case 2) RR:** 노드 z 의 오른쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 z 에서의 “Left Rotation”을 통해 균형 맞춤



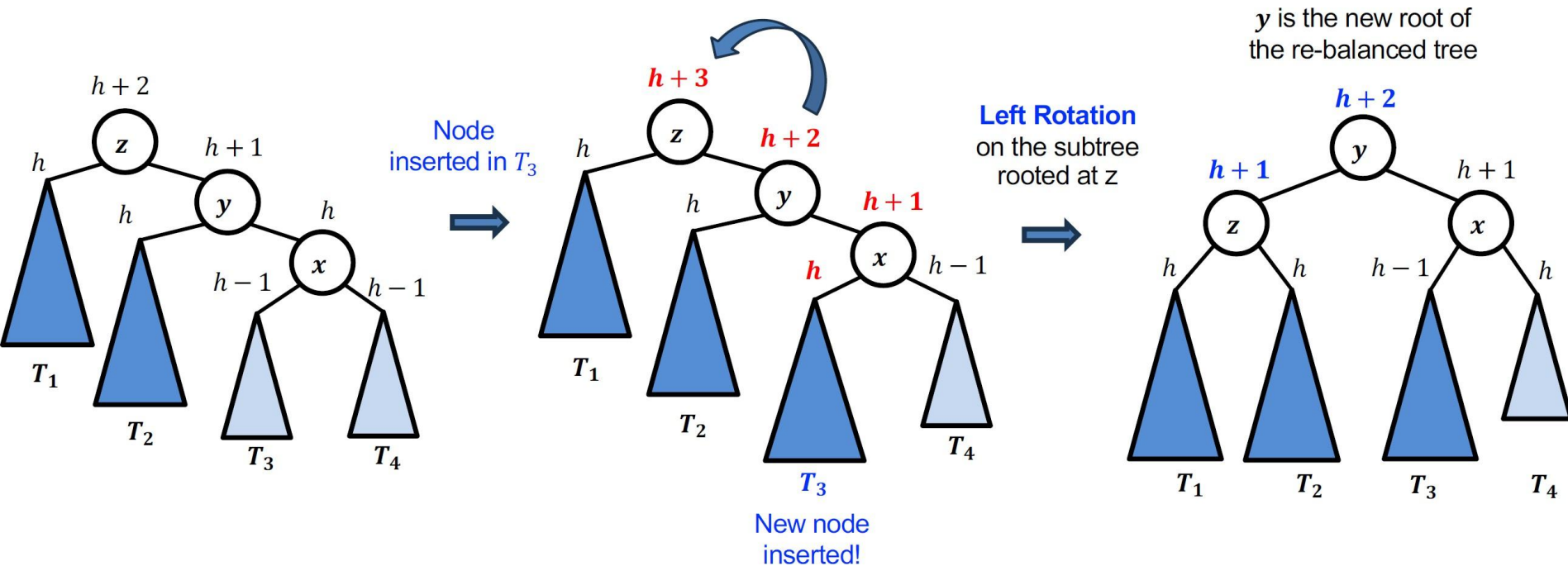
Insertion on AVL Tree (Case 2)

- **(Case 2) RR:** 노드 z 의 오른쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 z 에서의 “Left Rotation”을 통해 균형 맞춤



Insertion on AVL Tree (Case 2)

- **(Case 2) RR:** 노드 z 의 오른쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 z 에서의 “Left Rotation”을 통해 균형 맞춤

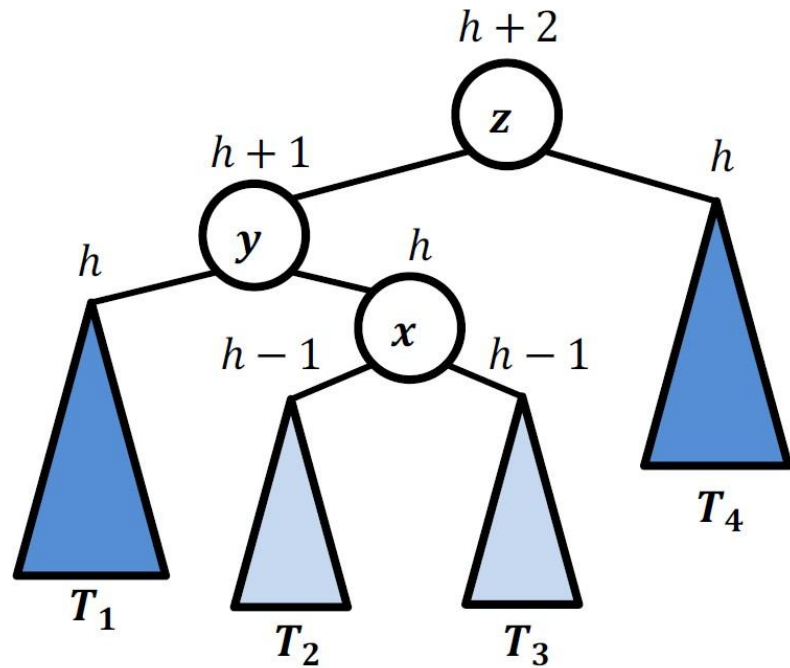


Left Rotation:

노드 z 가 y 의 왼쪽 자식 노드가 되며,
 y 의 기존 왼쪽 자식 노드는 z 의 오른쪽 자식 노드가 됨

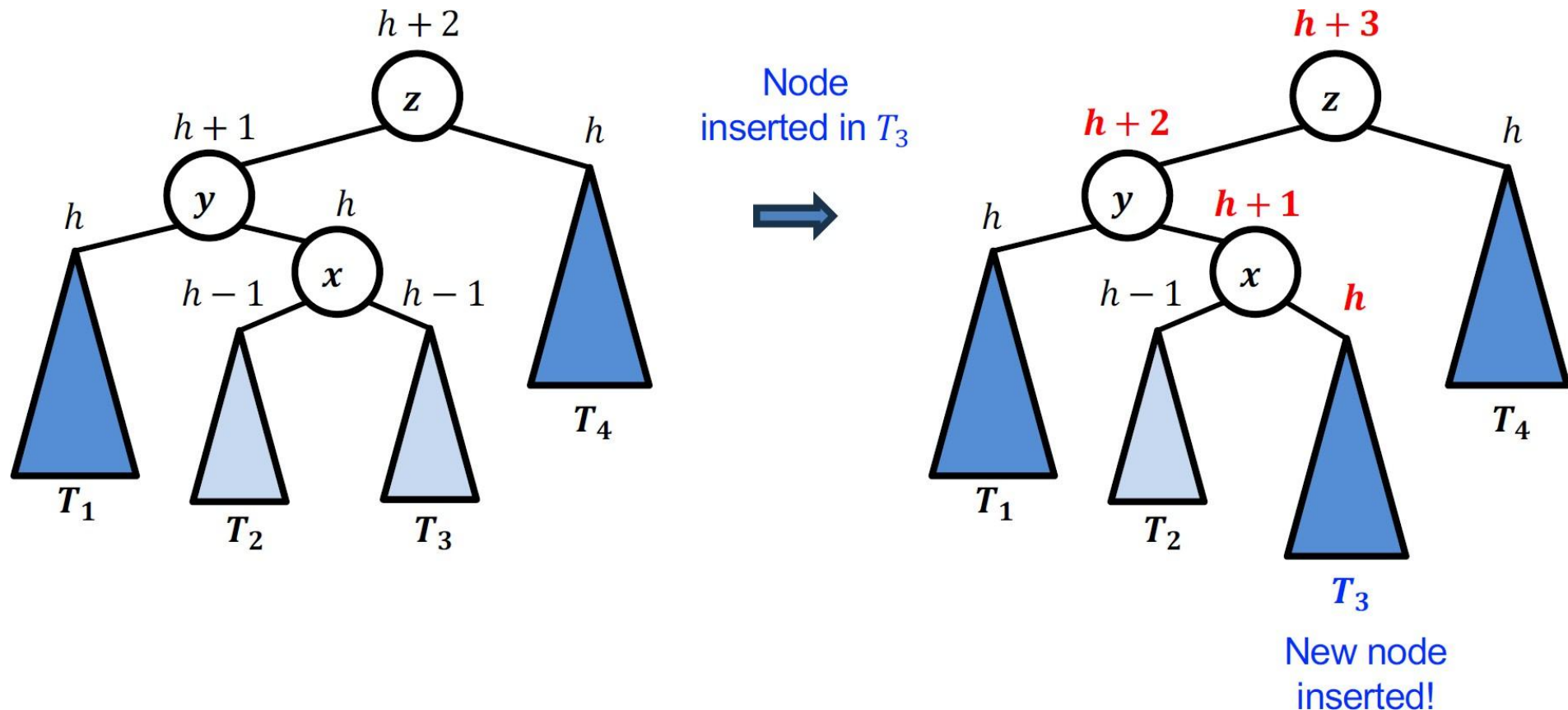
Insertion on AVL Tree (Case 3)

- **(Case 3) LR:** 노드 z 의 왼쪽 자식의 오른쪽 서브트리 내 삽입



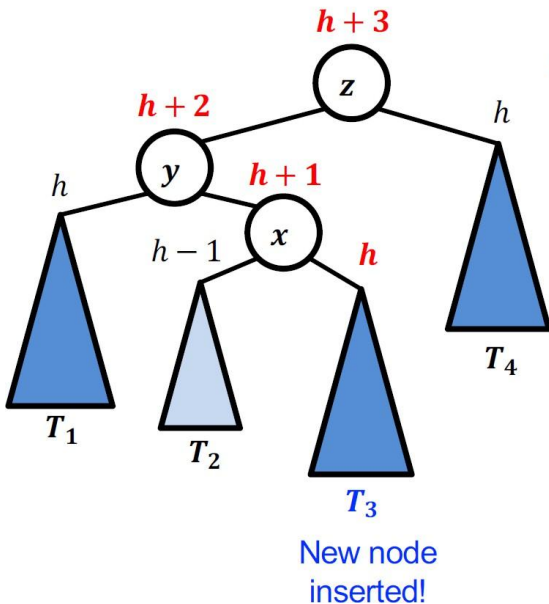
Insertion on AVL Tree (Case 3)

- (Case 3) LR: 노드 z 의 왼쪽 자식의 오른쪽 서브트리 내 삽입



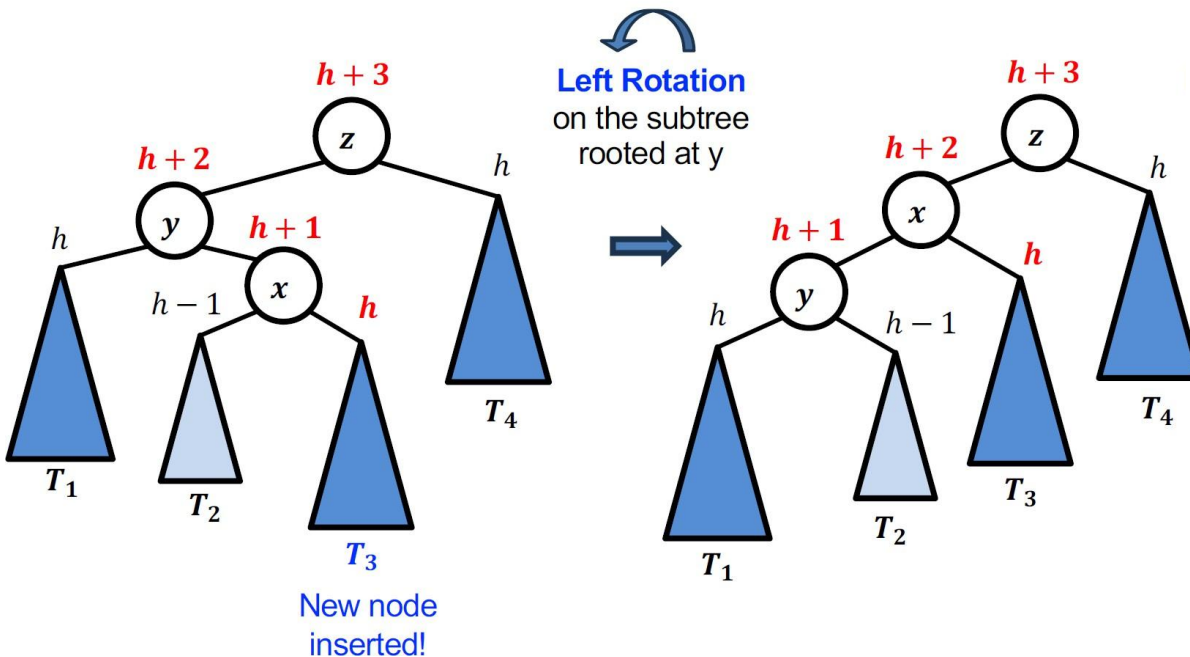
Insertion on AVL Tree (Case 3)

- **(Case 3) LR:** 노드 z 의 왼쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 y 에서의 “Left Rotation” 이후, z 에서의 “Right Rotation”을 통해 균형 맞춤



Insertion on AVL Tree (Case 3)

- **(Case 3) LR:** 노드 z 의 왼쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 y 에서의 “Left Rotation” 이후, z 에서의 “Right Rotation”을 통해 균형 맞춤

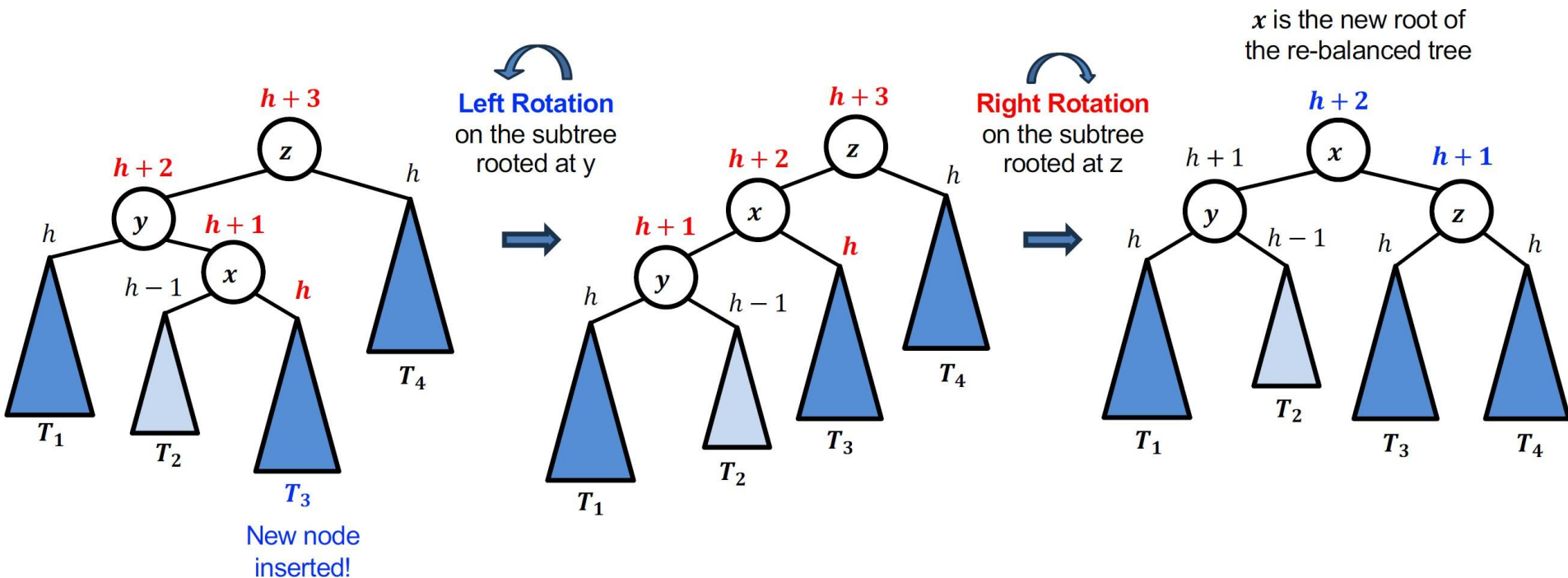


Left Rotation:

노드 y 가 x 의 왼쪽 자식 노드가 되며,
 x 의 기존 왼쪽 자식 노드는 y 의 오른쪽 자식 노드가 됨

Insertion on AVL Tree (Case 3)

- **(Case 3) LR:** 노드 z 의 왼쪽 자식의 오른쪽 서브트리 내 삽입
 - 노드 y 에서의 “Left Rotation” 이후, z 에서의 “Right Rotation”을 통해 균형 맞춤



Left Rotation:

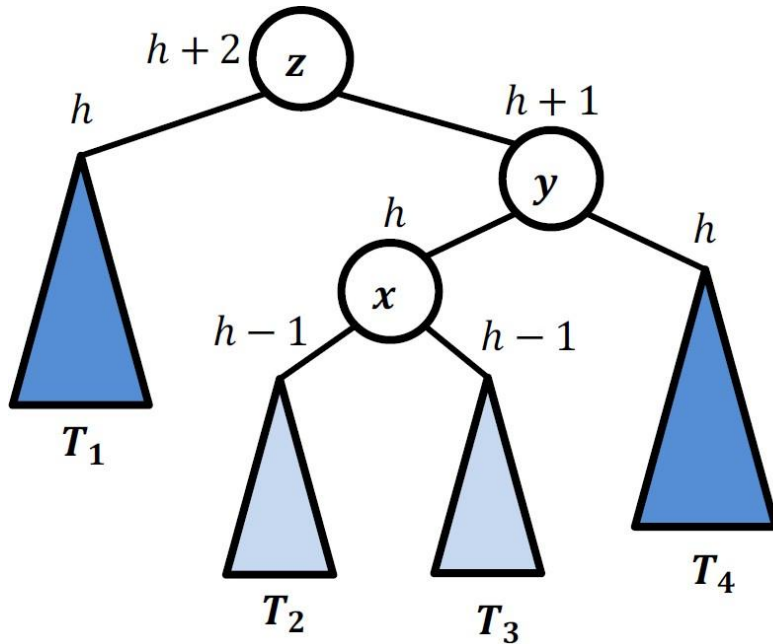
노드 y 가 x 의 왼쪽 자식 노드가 되며,
 x 의 기존 왼쪽 자식 노드는 y 의 오른쪽 자식 노드가 됨

Right Rotation:

노드 z 가 x 의 오른쪽 자식 노드가 되며,
 x 의 기존 오른쪽 자식 노드는 z 의 왼쪽 자식 노드가 됨

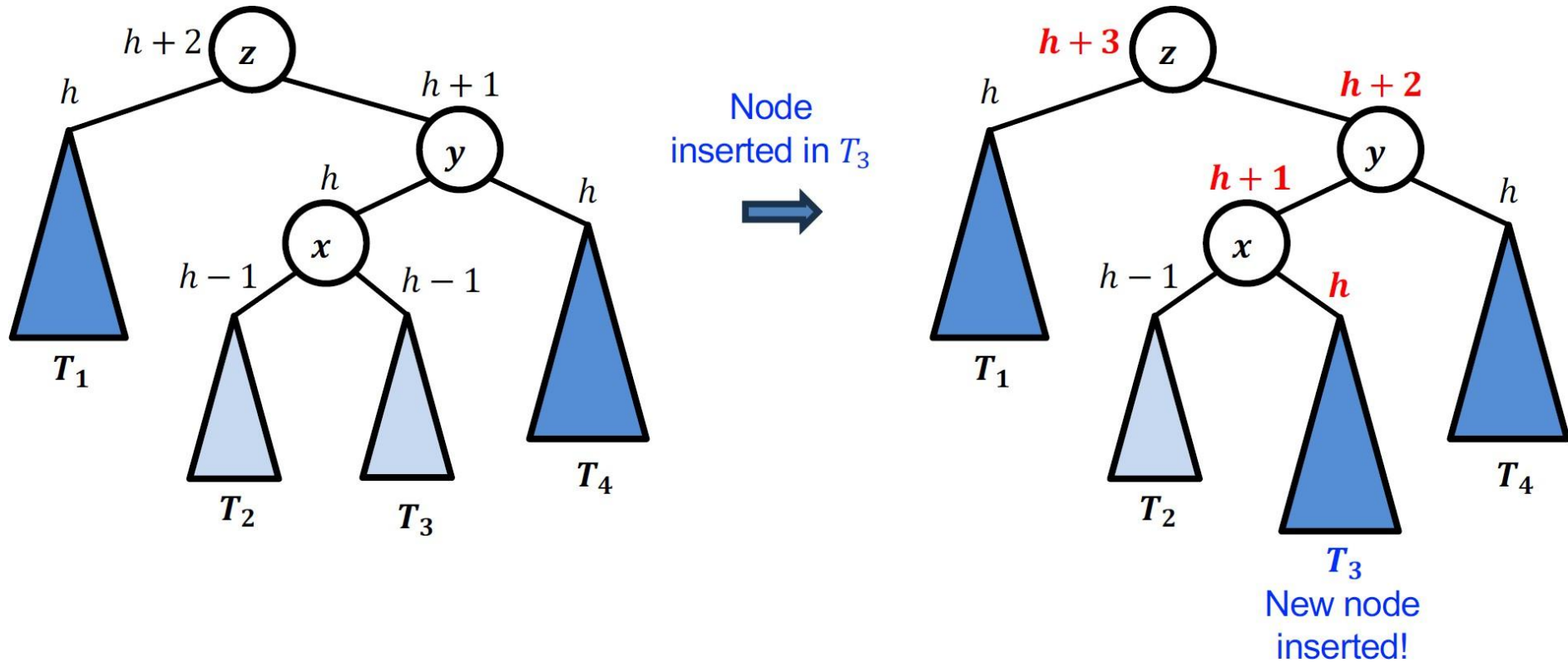
Insertion on AVL Tree (Case 4)

- (Case 4) RL: 노드 z 의 오른쪽 자식의 왼쪽 서브트리 내 삽입



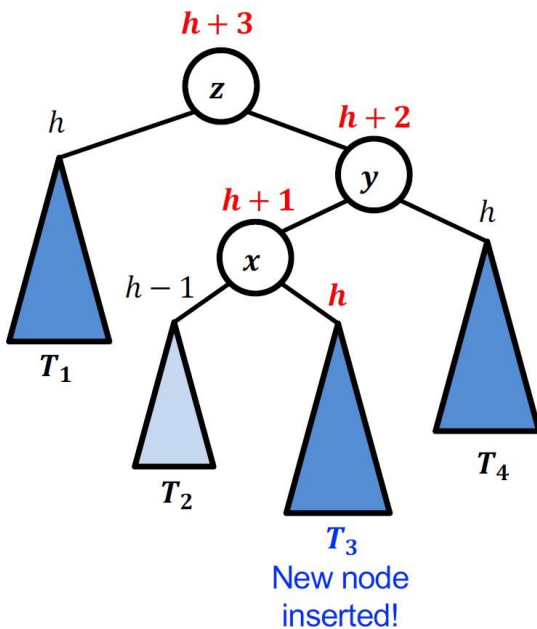
Insertion on AVL Tree (Case 4)

- (Case 4) RL: 노드 z 의 오른쪽 자식의 왼쪽 서브트리 내 삽입



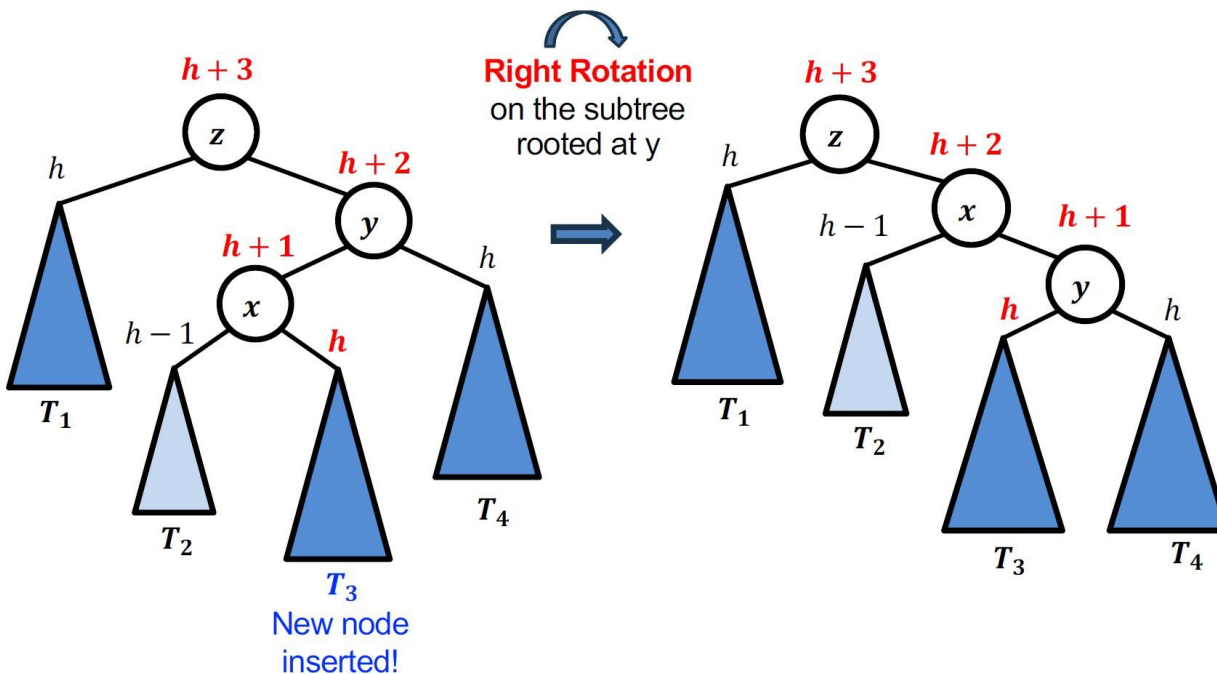
Insertion on AVL Tree (Case 4)

- **(Case 4) RL:** 노드 z 의 오른쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 y 에서의 “Right Rotation” 이후, z 에서의 “Left Rotation”을 통해 균형 맞춤



Insertion on AVL Tree (Case 4)

- **(Case 4) RL:** 노드 z 의 오른쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 y 에서의 “Right Rotation” 이후, z 에서의 “Left Rotation”을 통해 균형 맞춤

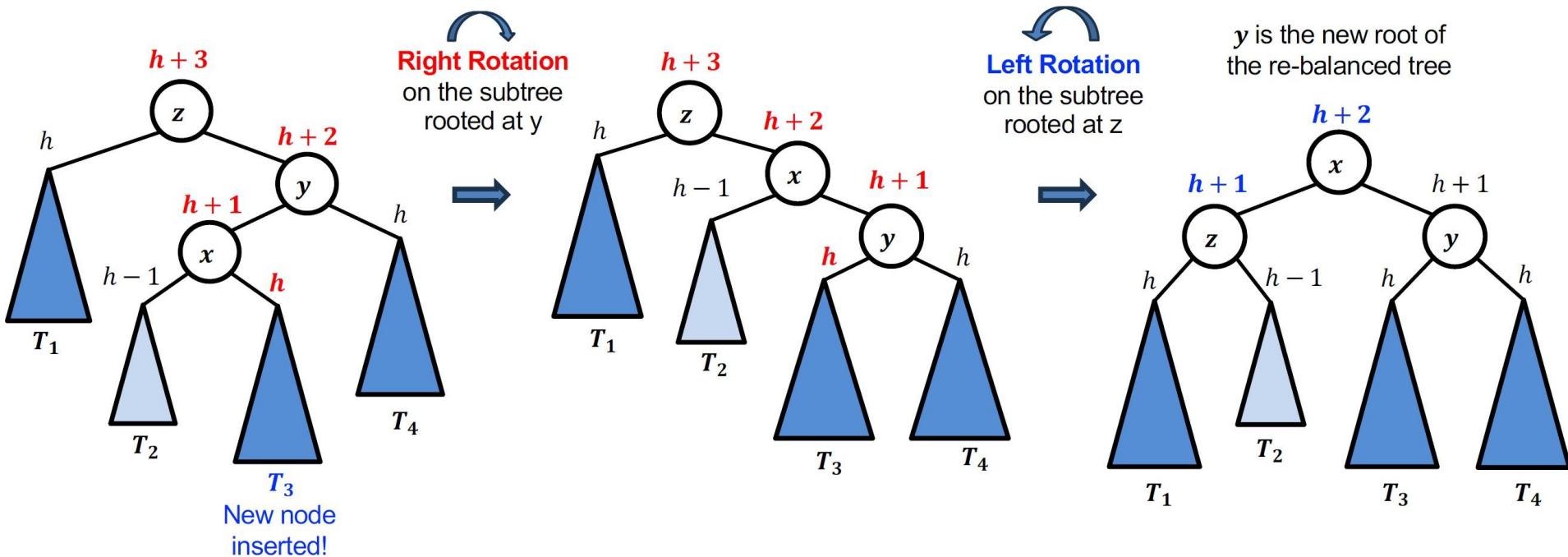


Right Rotation:

노드 y 가 x 의 오른쪽 자식 노드가 되며,
 x 의 기존 오른쪽 자식 노드는 y 의 왼쪽 자식 노드가 됨

Insertion on AVL Tree (Case 4)

- **(Case 4) RL:** 노드 z 의 오른쪽 자식의 왼쪽 서브트리 내 삽입
 - 노드 y 에서의 “Right Rotation” 이후, z 에서의 “Left Rotation”을 통해 균형 맞춤



Right Rotation:

노드 y 가 x 의 **오른쪽** 자식 노드가 되며,
 x 의 기존 오른쪽 자식 노드는 y 의 왼쪽 자식 노드가 됨

Left Rotation:

노드 z 가 x 의 **왼쪽** 자식 노드가 되며,
 x 의 기존 왼쪽 자식 노드는 z 의 오른쪽 자식 노드가 됨

Look ahead

AVL Trees (cont'd)